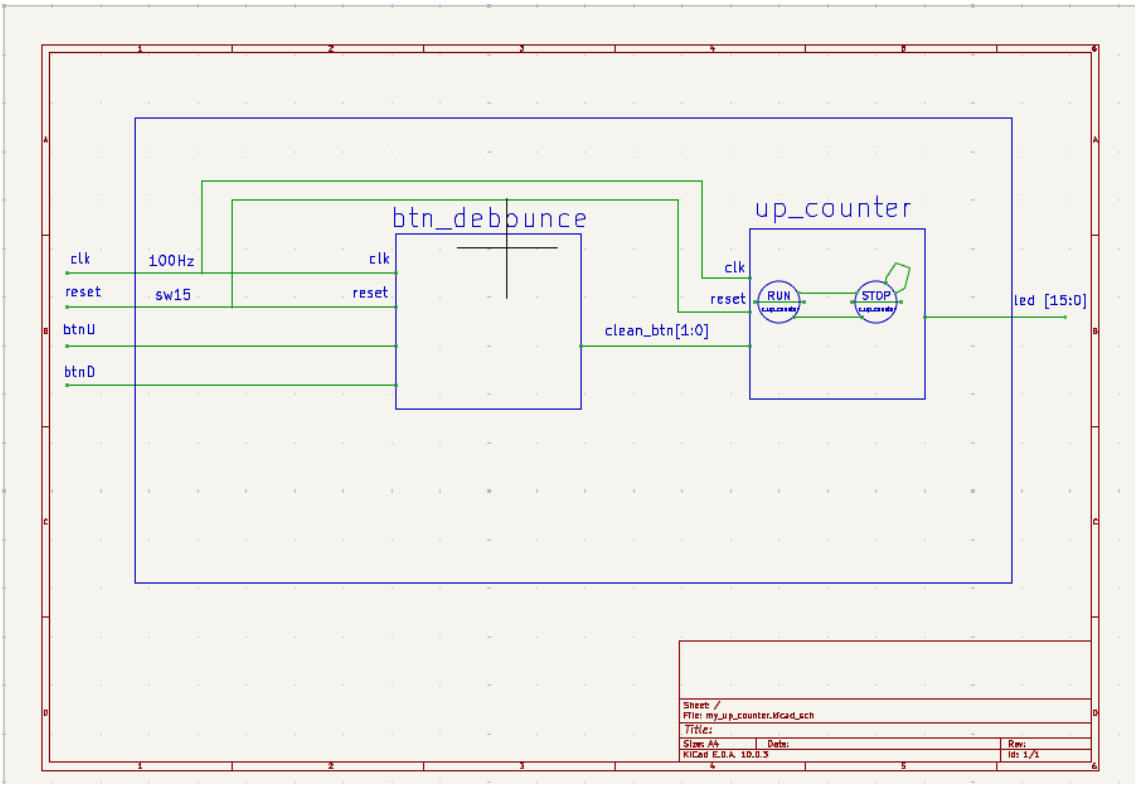
function_block_diagram



source code

top_my_up_counter.v

```
`timescale 1ns / 1ps

module top_my_up_counter(
    input clk,
    input reset,
    input btnU,
    input btnD,
    output [15:0]led
    );

    wire [1:0] w_clean_btn;

    btn_debounce u_btn_debounce_U(
        .clk(clk),
        .reset(reset),
        .btn(btnU),
        .clean_btn(w_clean_btn[0])
```

```
    );

    btn_debounce u_btn_debounce_D(
        .clk(clk),
        .reset(reset),
        .btn(btnD),
        .clean_btn(w_clean_btn[1])
    );

    up_counter u_up_counter(
        .clk(clk),
        .reset(reset),
        .clean_btn(w_clean_btn),
        .led(led)
    );
endmodule
```

btn_debounce.v

```
`timescale 1ns / 1ps

module btn_debounce(
    input clk,        // 100MHz
    input reset,
    input btn,
    output clean_btn
);

    reg [19:0] count = 0;
    reg btn_stable = 0;

    // 100MHz에서 10ms = 1,000,000클럭
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            count <= 0;
            btn_stable <= 0;
        end
        else begin
            if (btn == btn_stable) begin
```

```
                count <= 0;
            end
            else if (count == 999_999) begin
                btn_stable <= btn;
                count <= 0;
            end
            else begin
                count <= count + 1;
            end
        end
    end

    assign clean_btn = btn_stable;

endmodule
```

up_counter.v

```
`timescale 1ns / 1ps

module up_counter(
    input [1:0]clean_btn,
    input clk,
    input reset,
    output reg [15:0] led
    );

    reg [19:0] count = 0;
    reg [15:0] r_up_counter = 0;

    // 1이면 RUN 0이면 STOP
    reg state = 1;

    reg prev_btnU = 0;
    reg prev_btnD = 0;

    always @(posedge clk or posedge reset) begin
            if (reset) begin
            state <= 1;
            count <= 0;
```

```
            prev_btnU <= 0;
            prev_btnD <= 0;
            r_up_counter <= 0;
            led <= 0;
        end
            else begin
            prev_btnU <= clean_btn[0];
            prev_btnD <= clean_btn[1];

            if (state == 1) begin
                if (clean_btn[0] && !prev_btnU) begin
                    state <= 0;
                end
                if (count == 999_999) begin
                    count <= 0;
                    r_up_counter <= r_up_counter+1;
                    led <= led+1;
                end
            else begin
                    count <= count + 1;
                end
            end
            else begin
                count <= 0;
                if (clean_btn[0] && !prev_btnU) begin
                    state <= 1;
                end

                else if (clean_btn[1] && !prev_btnD) begin
                count <= 0;
                led <= 0;
                r_up_counter <= 0;
            end
            end
            end
    end
endmodule
```